

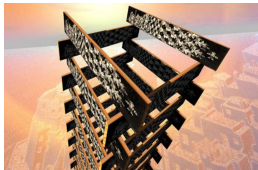
Produits réduits, domaines disjonctifs

TAS : Typage et analyse statique
M2, Master STL, Sorbonne Université

Antoine Miné

Année 2024–2025

Cours 11
20 décembre 2024



- combinaison de domaines abstraits :

- produit réduit

- ⇒ construction modulaire d'analyseurs
par combinaison de briques d'abstraction

- domaines disjonctifs :

- transformateurs de domaines génériques
permettant de représenter des ensembles non convexes

- complétion disjonctive
 - partitionnement d'états
 - partitionnement de traces

- conseils d'implantation pour le projet

Produits réduits

Principe

Théorie :

- l'ensemble des domaines abstraits forme un **treillis**,
- ordonné par la **notion d'abstraction**, qui est un ordre partiel où $(C, \leq) \xleftrightarrow[\alpha]{\gamma} (A, \sqsubseteq)$ indique que C est plus concret que A , i.e., toute propriété de A peut être représentée dans C ;
- il existe alors un plus petit majorant $\sqcup$ de domaines et un **plus grand minorant** $\sqcap$ de domaines.

Application : produit réduit

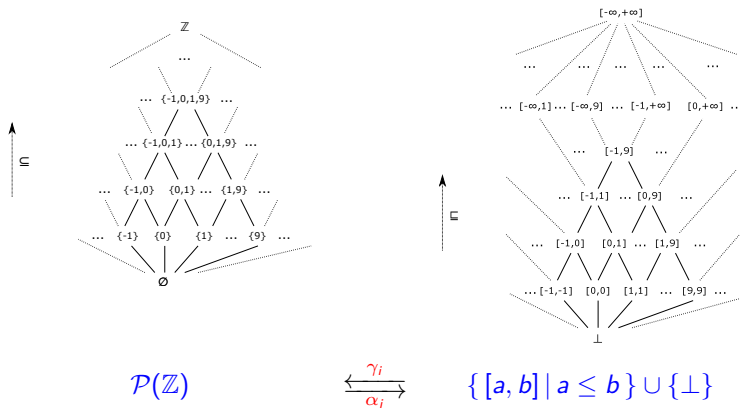
Construction effective du plus grand minorant $A_1 \sqcap A_2$ de deux abstractions, capable de représenter toutes les propriétés représentables dans A_1 **ou** dans A_2 .

Conséquence

Une analyse statique **précise** peut être construite en **combinant** des abstractions élémentaires.

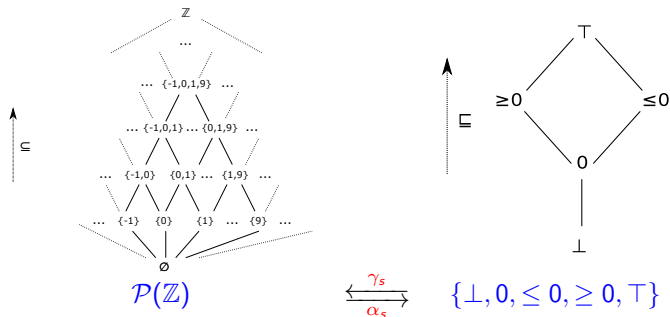
Treillis des abstractions

Rappel : abstraction des intervalles



- $\alpha_i(S) \stackrel{\text{def}}{=} [\min S, \max S]$
- $\gamma_i([a, b]) \stackrel{\text{def}}{=} \{x \in \mathbb{Z} \mid a \leq x \leq b\}$

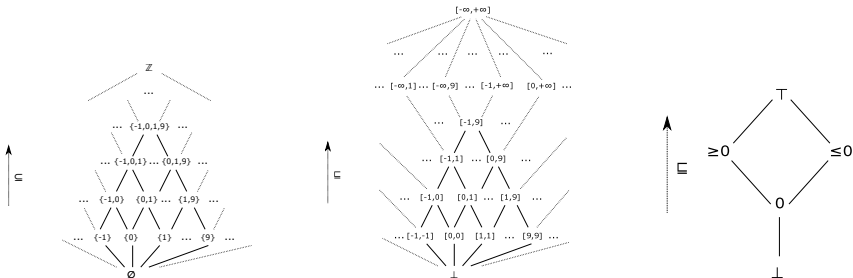
Rappel : abstraction des signes



$$\begin{array}{lll}
 \gamma_s(\perp) & \stackrel{\text{def}}{=} & \emptyset \\
 \gamma_s(0) & \stackrel{\text{def}}{=} & \{0\} \\
 \gamma_s(\geq 0) & \stackrel{\text{def}}{=} & \mathbb{N} \\
 \gamma_s(\leq 0) & \stackrel{\text{def}}{=} & -\mathbb{N} \\
 \gamma_s(\top) & \stackrel{\text{def}}{=} & \mathbb{Z}
 \end{array}$$

$$\alpha_s(S) \stackrel{\text{def}}{=} \begin{cases} \perp & \text{si } S = \emptyset \\ 0 & \text{si } S = \{0\} \\ \geq 0 & \text{sinon, si } \forall s \in S, s \geq 0 \\ \leq 0 & \text{sinon, si } \forall s \in S, s \leq 0 \\ \top & \text{sinon} \end{cases}$$

Composition d'abstractions



$$\mathcal{P}(\mathbb{Z}) \xrightleftharpoons[\alpha_i]{\gamma_i} \{[a, b] \mid a \leq b\} \cup \{\perp\} \xrightleftharpoons[\alpha'_s]{\gamma'_s} \{\perp, 0, \leq 0, \geq 0, T\}$$

$$\text{où } \gamma'_s(\perp) \stackrel{\text{def}}{=} \perp \quad \gamma'_s(T) \stackrel{\text{def}}{=} [-\infty, +\infty]$$

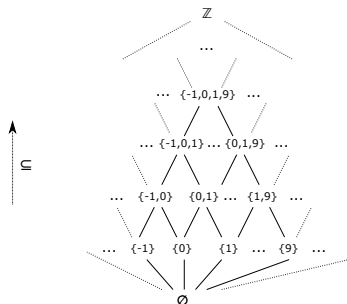
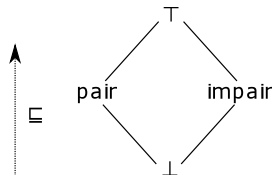
$$\gamma'_s(\geq 0) \stackrel{\text{def}}{=} [0, +\infty] \quad \gamma'_s(\leq 0) \stackrel{\text{def}}{=} [-\infty, 0] \quad \gamma'_s(0) \stackrel{\text{def}}{=} [0, 0]$$

Composition d'abstractions : les correspondances de Galois se composent

$$\text{Si } (X_1, \sqsubseteq_1) \xrightleftharpoons[\alpha_1]{\gamma_1} (X_2, \sqsubseteq_2) \xrightleftharpoons[\alpha_2]{\gamma_2} (X_3, \sqsubseteq_3), \text{ alors } (X_1, \sqsubseteq_1) \xrightleftharpoons[\alpha_2 \circ \alpha_1]{\gamma_1 \circ \gamma_2} (X_3, \sqsubseteq_3).$$

$$\text{preuve : } (\alpha_2 \circ \alpha_1)(c) \sqsubseteq_3 a \iff \alpha_1(c) \sqsubseteq_2 \gamma_2(a) \iff c \sqsubseteq_1 (\gamma_1 \circ \gamma_2)(a)$$

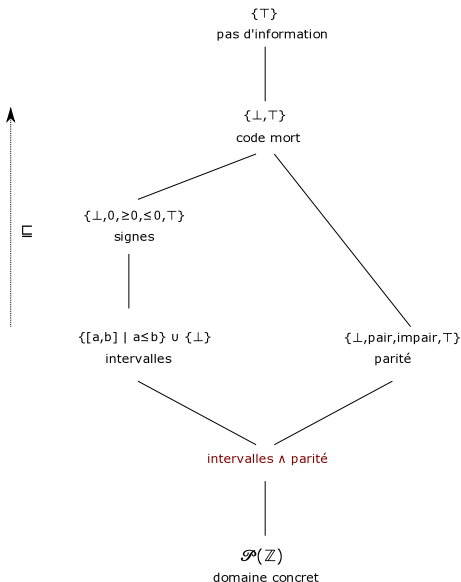
Un nouveau domaine simple : le domaine des parités


 $\mathcal{P}(\mathbb{Z})$

 $\{\perp, \top, \text{pair}, \text{impair}\}$

$$\begin{array}{lll}
 \gamma_p(\perp) & \stackrel{\text{def}}{=} & \emptyset \\
 \gamma_p(\text{pair}) & \stackrel{\text{def}}{=} & 2\mathbb{Z} \\
 \gamma_p(\text{impair}) & \stackrel{\text{def}}{=} & 2\mathbb{Z} + 1 \\
 \gamma_p(\top) & \stackrel{\text{def}}{=} & \mathbb{Z}
 \end{array}$$

$$\alpha_p(S) \stackrel{\text{def}}{=} \begin{cases} \perp & \text{si } S = \emptyset \\ \text{pair} & \text{sinon, si } S \subseteq 2\mathbb{Z} \\ \text{impair} & \text{sinon, si } S \subseteq 2\mathbb{Z} + 1 \\ \top & \text{sinon} \end{cases}$$

Treillis des abstractions de $\mathcal{P}(\mathbb{Z})$



- les signes sont plus abstraits que les intervalles ;
toutes propriété de signe est exprimable comme un intervalle
- les parités et les intervalles sont incomparables ;
aucune propriété commune, excepté $\perp$ et T
- $\mathcal{P}(\mathbb{Z})$ est le domaine le plus concret ;
- $\{T\}$ est le domaine le plus abstrait ;
- **intervalles** $\wedge$ **parité** est le domaine le plus abstrait à la fois plus précis que les intervalles et plus précis que les parités.

Construction du produit réduit

Produit simple de domaines

Structure :

Étant donnés deux domaines $(\mathcal{D}_1^\sharp, \sqsubseteq_1)$ et $(\mathcal{D}_2^\sharp, \sqsubseteq_2)$, nous représentons dans $\mathcal{D}^\sharp$ les **paire**s d'éléments abstraits ; elles correspondent à des **conjonctions** de propriétés.

- $\mathcal{D}_{1 \times 2}^\sharp \stackrel{\text{def}}{=} \mathcal{D}_1^\sharp \times \mathcal{D}_2^\sharp$
- $\gamma_{1 \times 2}(X_1^\sharp, X_2^\sharp) \stackrel{\text{def}}{=} \gamma_1(X_1^\sharp) \cap \gamma_2(X_2^\sharp)$
- $\alpha_{1 \times 2}(S) \stackrel{\text{def}}{=} (\alpha_1(S), \alpha_2(S))$
- $(X_1^\sharp, X_2^\sharp) \sqsubseteq_{1 \times 2} (Y_1^\sharp, Y_2^\sharp) \stackrel{\text{def}}{\iff} X_1^\sharp \sqsubseteq_1 Y_1^\sharp \text{ and } X_2^\sharp \sqsubseteq_2 Y_2^\sharp$

Opérations abstraites dans $\mathcal{D}^\sharp$:

Effectuées en parallèle sur chaque élément, dans son domaine respectif :

- $(X_1^\sharp, X_2^\sharp) \cup_{1 \times 2}^\sharp (Y_1^\sharp, Y_2^\sharp) \stackrel{\text{def}}{=} (X_1^\sharp \cup_1^\sharp Y_1^\sharp, X_2^\sharp \cup_2^\sharp Y_2^\sharp), ;$
- $(X_1^\sharp, X_2^\sharp) \nabla_{1 \times 2}^\sharp (Y_1^\sharp, Y_2^\sharp) \stackrel{\text{def}}{=} (X_1^\sharp \nabla_1^\sharp Y_1^\sharp, X_2^\sharp \nabla_2^\sharp Y_2^\sharp) ;$
- $S^\sharp \llbracket s \rrbracket_{1 \times 2}(X_1^\sharp, X_2^\sharp) \stackrel{\text{def}}{=} (S^\sharp \llbracket s \rrbracket_1(X_1^\sharp), S^\sharp \llbracket s \rrbracket_2(X_2^\sharp)).$

Produit simple d'analyses : limitation

```

V ← 1;
while V ≤ 10 do V ← V + 2 done;
● if V ≥ 12 then ● V ← 0 ●;

```

Analyse dans le domaine des intervalles, des parités, et leur produit :

	intervalles	parités	produit : intervalles×parités
●	$V \in [11, 12]$	V impair	$(V \in [11, 12], V \text{ impair})$
●	$V = 12$	V impair	$(V = 12, V \text{ impair})$
●	$V = 0$	V pair	$(V = 0, V \text{ pair})$

L'analyse est identique à deux **analyses séparées** :

- en ●, elle trouve $V = 12$ et V impair qui est insatisfiable, i.e., $\emptyset$;
- en ●, l'application indépendante de $V \leftarrow 0$ sur les intervalles et les parités redonne $V = 0$.

$\gamma_{1 \times 2}(S^\sharp \llbracket V \leftarrow 0 \rrbracket (X^\sharp, Y^\sharp)) \neq \emptyset$ alors que $\gamma_{1 \times 2}(X^\sharp, Y^\sharp) = \emptyset$!

⇒ perte de précision importante.

Solution : produit totalement réduit

Principe : propager des informations entre les domaines

Étant données les correspondances de Galois (α_1, γ_1) et (α_2, γ_2) sur $\mathcal{D}_1^\sharp$ et $\mathcal{D}_2^\sharp$, nous définissons l'**opérateur de réduction** ρ comme :

$$\begin{aligned} \rho : \mathcal{D}_{1 \times 2}^\sharp &\rightarrow \mathcal{D}_{1 \times 2}^\sharp \\ \rho(X_1^\sharp, X_2^\sharp) &\stackrel{\text{def}}{=} (\alpha_1(\gamma_1(X_1^\sharp) \cap \gamma_2(X_2^\sharp)), \alpha_2(\gamma_1(X_1^\sharp) \cap \gamma_2(X_2^\sharp))) \end{aligned}$$

i.e., la meilleure représentation de $\gamma_{1 \times 2}(X_1^\sharp, X_2^\sharp)$ dans chacun des domaines.

Application :

Utilisation de ρ pour affiner le résultat **après chaque opération abstraite** :

- $(X_1^\sharp, X_2^\sharp) \cup_{1 \times 2}^\sharp (Y_1^\sharp, Y_2^\sharp) \stackrel{\text{def}}{=} \rho(X_1^\sharp \cup_1^\sharp Y_1^\sharp, X_2^\sharp \cup_2^\sharp Y_2^\sharp),$
- $S^\sharp \llbracket s \rrbracket_{1 \times 2}(X_1^\sharp, X_2^\sharp) \stackrel{\text{def}}{=} \rho(S^\sharp \llbracket s \rrbracket_1(X_1^\sharp), S^\sharp \llbracket s \rrbracket_2(X_2^\sharp)).$

Attention :

ne pas appliquer ρ après ∇ ; cela pourrait empêcher la convergence des itérés dans $\mathcal{D}_1^\sharp \times \mathcal{D}_2^\sharp$!
intuition : ∇ accélère la croissance, tandis que ρ la limite.

Exemple d'analyse avec réduction

```

V ← 1;
while V ≤ 10 do V ← V + 2 done;
• if V ≥ 12 then V ← 0
  
```

Réduction ρ entre les intervalles et les parités $\rho([a, b], p)$:

D'abord raffiner les bornes de l'intervalle $[a, b]$ en utilisant la parité p , puis utiliser les bornes pour raffiner la parité :

- soit $a' = a + 1$ si $a \notin \gamma_p(p)$, $a' = a$ sinon ;
- soit $b' = b - 1$ si $b \notin \gamma_p(p)$, $b' = b$ sinon ;
- si $a' > b'$, retourner $(\perp, \perp)$;
- si $a' = b'$, retourner $([a', a'], \alpha_p(a'))$;
- sinon, retourner $([a', b'], p)$.

Exemple :

En •, $\rho([11, 12], \text{impair}) = ([11, 11], \text{impair})$
 $\implies$ la branche **then** n'est pas atteignable.

Limitations de la réduction totale, réduction partielle

La réduction ρ optimale est bien définie mathématiquement, mais :

- ρ suppose l'existence de **correspondances de Galois** ;
- la formule ne donne **pas d'algorithme effectif** pour implanter ρ .

cas similaire à l'abstraction optimale d'un opérateur : $F^\# \stackrel{\text{def}}{=} \alpha \circ F \circ \gamma$

Réduction partielle :

Définition pragmatique, quand la réduction optimale n'est pas utilisable :

- $\rho(X_1^\#, X_2^\#) = (Y_1^\#, Y_2^\#)$ est une réduction partielle si :
- $Y_1^\# \sqsubseteq_1 X_1^\#$ et $Y_2^\# \sqsubseteq_2 X_2^\#$; (raffinement)
- $\gamma_{1 \times 2}(Y_1^\#, Y_2^\#) = \gamma_{1 \times 2}(X_1^\#, X_2^\#)$. (sûreté)

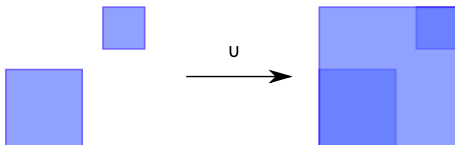
Exemple générique :

$$\rho(X_1^\#, X_2^\#) \stackrel{\text{def}}{=} \begin{cases} (\perp_1, \perp_2) & \text{si } X_1^\# = \perp_1 \text{ ou } X_2^\# = \perp_2 \\ (X_1^\#, X_2^\#) & \text{sinon} \end{cases}$$

En pratique, un analyseur comporte de nombreux domaines abstraits (expressivité) avec des réductions très limitées entre domaines (efficacité).

Domaines disjonctifs

Perte de précision : abstraction de l'union



Perte de précision :

Les domaines sont **rarement clos par union** :

- pour les constantes et les parités , $a \cup^\# b = \top$ si $a \neq b$;
- les intervalles ne représentent que des **ensembles convexes** ;

$\Rightarrow$ perte de précision à chaque application de $\cup^\#$.

Particulièrement gênant car $\cup^\#$ est utilisé fréquemment :

- $C^\# \llbracket e_1 \vee e_2 \rrbracket$ (ou logique dans un test)
- $S^\# \llbracket \text{if } c \text{ then } s_1 \text{ else } s_2 \rrbracket$ (fusion des branches **then** et **else**)
- $S^\# \llbracket \text{while } c \text{ do } s \rrbracket$ (fusion des itérés pour inférer un invariant de boucle)

Motivation

Remarque : la plus part des domaines représentent des **ensembles convexes**

(conjonctions de contraintes convexes)

$\Rightarrow \cup^\#$ cause des pertes de précision importantes !

Utilité des invariants non convexes

```

X ← rand(10, 20);
Y ← rand(0, 1);
if Y > 0 then X ← -X;
• Z ← 100/X
  
```

Sémantique concrète :

En •, $X \in [-20, -10] \cup [10, 20]$

$\Rightarrow$ pas de **division par zéro**

Analyse abstraite :

Toute analyse convexe (intervalles, polyèdres) trouvera $X \in [-20, 20]$

dans les intervalles : $[-20, -10] \cup^\# [10, 20] = [-20, 20]$

$\Rightarrow$ **division par zéro possible** (fausse alarme)

Domaines disjonctifs

Domaines disjonctifs :

constructions génériques pour étendre tout domaine abstrait numérique en un domaine représentant certaines disjonctions de manière exacte.

Principe :

utiliser plusieurs éléments abstraits à chaque point de programme au lieu d'un seul.

Plusieurs types de constructions possibles :

- complétion par l'ensemble des parties

« soupe » non structurée d'éléments abstraits

- partitionnement d'état

fixer un partitionnement de $\mathcal{E}$, utiliser un élément abstrait par élément de partition

- analyses sensibles aux chemins d'exécution (*path-sensitive*)

partitionnement vis à vis de l'historique des exécutions

chaque construction a ses forces et ses faiblesses ;
souvent, plusieurs constructions sont combinées dans une analyse

Complétion par l'ensemble des parties finies

Domaine de l'ensemble des parties finies

Étant donné : $(\mathcal{E}^\#, \sqsubseteq, \gamma, \cup^\#, \cap^\#, \nabla, S^\# \llbracket stat \rrbracket)$

un domaine abstrait $\mathcal{E}^\#$

ordonné par $\sqsubseteq$

avec une concrétisation $\gamma : \mathcal{E}^\# \rightarrow \mathcal{P}(\mathcal{E})$

des abstractions sûres $\cup^\#, \cap^\#, S^\# \llbracket stat \rrbracket$ de $\cup, \cap, S \llbracket stat \rrbracket$

et un élargissement ∇

Le domaine des parties est : $(\hat{\mathcal{E}}^\#, \hat{\sqsubseteq}, \hat{\gamma}, \hat{\cup}^\#, \hat{\cap}^\#, \hat{\nabla}, \hat{S}^\# \llbracket stat \rrbracket)$

$$\blacksquare \hat{\mathcal{E}}^\# \stackrel{\text{def}}{=} \mathcal{P}_{finis}(\mathcal{E}^\#)$$

ensembles finis d'éléments abstraits

$$\blacksquare \hat{\gamma}(A^\#) \stackrel{\text{def}}{=} \cup \{ \gamma(X^\#) \mid X^\# \in A^\# \}$$

union ensembliste des concrétisations

Exemple : avec le domaine des intervalles $\mathcal{E}^\#$

$$\hat{\gamma}(\{[-10, -5], [2, 4], [0, 0], [2, 3]\}) = [-10, -5] \cup \{0\} \cup [2, 4]$$

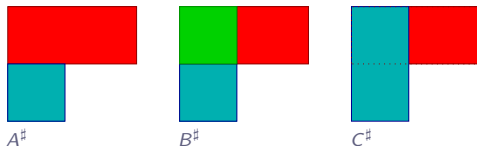
Ordre

Problème : comment comparer deux éléments de $\widehat{\mathcal{E}}^\#$?

- $\widehat{\gamma}$ n'est généralement pas injective
il n'y a pas de représentation canonique de $\widehat{\gamma}(A^\#)$
- tester $\widehat{\gamma}(A^\#) = \widehat{\gamma}(B^\#)$ ou $\widehat{\gamma}(A^\#) \subseteq \widehat{\gamma}(B^\#)$ est généralement très difficile !

$\implies$ nous utilisons une solution approchée.

Exemple : ensemble des parties sur le domaine des intervalles



$$A^\# = \{\{0\} \times \{0\}, [0, 1] \times \{1\}\}$$

$$B^\# = \{\{0\} \times \{0\}, \{0\} \times \{1\}, \{1\} \times \{1\}\}$$

$$C^\# = \{\{0\} \times [0, 1], [0, 1] \times \{1\}\}$$

$$\widehat{\gamma}(A^\#) = \widehat{\gamma}(B^\#) = \widehat{\gamma}(C^\#)$$

$B^\#$ est plus coûteux à représenter : il nécessite trois éléments abstraits au lieu de deux ;

$C^\#$ est un recouvrement et non une partition (rouge $\cap$ bleu = $\{0\} \times \{1\} \neq \emptyset$)

Ordre (suite)

Solution : abstraction sûre de $\subseteq$

$$A^\# \hat{\subseteq} B^\# \stackrel{\text{def}}{\iff} \forall X^\# \in A^\# : \exists Y^\# \in B^\# : X^\# \subseteq Y^\#$$

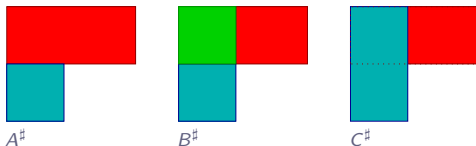
tout élément de $A^\#$ est inclus dans un élément de $B^\#$

relation de Hoare

- $\hat{\subseteq}$ est un ordre partiel, en supposant que $\subseteq$ en soit un
- $\hat{\subseteq}$ est une approximation de $\subseteq$, si $\subseteq$ en est aussi une
 $A^\# \hat{\subseteq} B^\# \implies \hat{\gamma}(A^\#) \subseteq \hat{\gamma}(B^\#)$ mais la réciproque n'est pas forcément vraie !
- tester $\hat{\subseteq}$ se réduit à tester $\subseteq$ un nombre fini de fois

$\implies$ très efficace !

Exemple : ensemble des parties du domaine des intervalles



$$\hat{\gamma}(A^\#) = \hat{\gamma}(B^\#) = \hat{\gamma}(C^\#)$$

$$B^\# \hat{\subseteq} A^\# \hat{\subseteq} C^\#$$

Opérations abstraites

Opérations abstraites :

- $\hat{S}^\# \llbracket \text{stat} \rrbracket A^\# \stackrel{\text{def}}{=} \{ S^\# \llbracket \text{stat} \rrbracket X^\# \mid X^\# \in A^\# \}$
appliquer $S^\# \llbracket \text{stat} \rrbracket$ indépendamment sur chaque élément de $\mathcal{E}^\#$
- $A^\# \hat{\cup}^\# B^\# \stackrel{\text{def}}{=} A^\# \cup B^\#$
garde intacts les éléments abstraits de chaque argument
sans appliquer $\cup^\#$, donc **sans** perte de précision
 $\hat{\cup}^\#$ est **exact** !
- $A^\# \hat{\cap}^\# B^\# \stackrel{\text{def}}{=} \{ X^\# \cap^\# Y^\# \mid X^\# \in A^\#, Y^\# \in B^\# \}$
 $\hat{\cap}^\#$ est toujours **exact** si $\cap^\#$ l'est
(car $\cap$ distribue $\cup$ dans le concret)

Correspondance de Galois :

en général, il n'y a pas de fonction d'abstraction $\hat{\alpha}$ correspondant à $\hat{\gamma}$

Exemple : ensemble des parties $\hat{\mathcal{E}}^\#$ sur le domaine des intervalles $\mathcal{E}^\#$

étant donné le disque $S \stackrel{\text{def}}{=} \{ (x, y) \mid x^2 + y^2 \leq 1 \}$
 $\alpha(S) = [-1, 1] \times [-1, 1]$ (abstraction optimale dans les intervalles)

mais il n'y pas de meilleur abstraction dans $\hat{\mathcal{E}}^\#$



$\alpha(S)$



pas de $\hat{\alpha}(S)$

Exemple d'analyse

Exemple

```

X ← rand(10, 20);
Y ← rand(0, 1);
• if Y > 0 then X ← -X;
• Z ← 100/X

```

Analyse :

- $\mathcal{E}^\#$ est le domaine des intervalles
 - en •, l'élément abstrait est $\{(X \in [10, 20], Y \in [0, 1])\}$
 - lors du test $\hat{S}^\# \llbracket \text{if } Y > 0 \text{ then } X \leftarrow -X \rrbracket$:
 - la branche **then** donne $\{(X \in [-20, -10], Y \in [1, 1])\}$
 - la branche **else** donne $\{(X \in [10, 20], Y \in [0, 0])\}$
 - l'union en • donne :

$$A^\# \stackrel{\text{def}}{=} \{(X \in [-20, -10], Y \in [1, 1])\} \hat{\cup} \{(X \in [10, 20], Y \in [0, 0])\}$$

$$= \{(X \in [-20, -10], Y \in [1, 1]), (X \in [10, 20], Y \in [0, 0])\}$$
 - ni $X \in [10, 20]$, ni $X \in [-20, -10]$ ne contient 0
- $\implies \hat{S}^\# \llbracket Z \leftarrow 100/X \rrbracket A^\#$ n'a pas de division par 0

Élargissement

Problème : les itérés $(A_n^\#)_{n \in \mathbb{N}}$ dans les boucles ne convergent pas forcément

- le nombre d'éléments dans $A_n^\#$ peut croître de manière arbitraire ;
- même si $|A_n^\#|$ est stable, les éléments de $A_n^\#$ ne convergent pas forcément ;
dans le cas où $\mathcal{E}^\#$ a des séquences infinies strictement croissantes

$\implies$ un **élargissement** ∇ est nécessaire.

La construction d'élargissements pour les domaines d'ensembles de parties est un **problème difficile**.

Exemple naïf d'élargissement :

fusionner les éléments abstraits à partir d'un nombre fixé d'itérations

$$A_{n+1}^\# \stackrel{\text{def}}{=} \begin{cases} A_n^\# \hat{\cup}^\# B_{n+1}^\# & \text{si } n < N \\ \{(\cup^\# A_n^\#) \nabla (\cup^\# B_{n+1}^\#)\} & \text{sinon} \end{cases}$$

Note : d'autres élargissements plus sophistiqués existent.

Partitionnement d'états

Partitionnement d'états

Principe :

- partitionner *a priori* $\mathcal{E}$ en un nombre fini, fixé de parties
- abstraire les états contenus dans chaque partie de manière indépendante par un élément abstrait de $\mathcal{E}^\#$

Domaine abstrait :

Étant donné un domaine abstrait $\mathcal{E}^\#$,
 nous choisissons une **partition de $\mathcal{E}$ représentable dans $\mathcal{E}^\#$** ;
 i.e., un ensemble $P \subseteq \mathcal{E}^\#$ tel que :

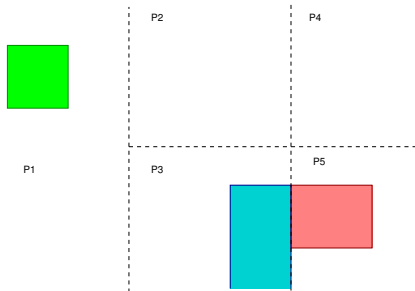
- P est fini : $P = \{ P_1, \dots, P_n \}$
- $\bigcup \{ \gamma(P_i) \mid i \in [1, n] \} = \mathcal{E}$

Note : il peut s'agir en réalité d'un **recouvrement** de $\mathcal{E}$ et non d'une partition ;
 nous autorisons : $\gamma(P_i) \cap \gamma(P_j) \neq \emptyset$

cela donne plus de flexibilité dans le choix de $\mathcal{E}^\#$ (e.g., pas forcément de contraintes strictes)

Le domaine partitionné est alors : $\tilde{\mathcal{E}}^\# \stackrel{\text{def}}{=} [1, n] \rightarrow \mathcal{E}^\#$
 représentable en mémoire car P est fini : tableau ou *map*

Structure d'ordre



Exemple : ici, $\mathcal{E}^\sharp$ est le domaine des intervalles

$P = \{P_1, P_2, P_3, P_4, P_5\}$ où

$P_1 = [-\infty, 0] \times [-\infty, +\infty]$

$P_2 = [0, 10] \times [0, +\infty]$

$P_3 = [0, 10] \times [-\infty, 0]$

$P_4 = [10, +\infty] \times [0, +\infty]$

$P_5 = [10, +\infty] \times [-\infty, 0]$

$X^\sharp = [P_1 \mapsto [-6, -5] \times [5, 6],$

$P_2 \mapsto \perp,$

$P_3 \mapsto [9, 10] \times [-\infty, -1],$

$P_4 \mapsto \perp,$

$P_5 \mapsto [10, 12] \times [-3, -1]]$

$$\blacksquare \tilde{\mathcal{E}}^\sharp \stackrel{\text{def}}{=} [1, n] \rightarrow \mathcal{E}^\sharp$$

à chaque élément de la partition $P_i \in P$ est associé un élément abstrait $A^\sharp(i)$

$$\blacksquare \tilde{\gamma}(A^\sharp) \stackrel{\text{def}}{=} \cup \{ \gamma(A^\sharp(i)) \cap \gamma(P_i) \mid i \in [1, n] \}$$

chaque $A^\sharp(i)$ représente un ensemble d'environnements inclus dans $\gamma(P_i)$

$$\blacksquare A^\sharp \sqsubseteq B^\sharp \stackrel{\text{def}}{\iff} \forall i \in [1, n]: A^\sharp(i) \subseteq B^\sharp(i)$$

ordre point à point

$$\blacksquare \tilde{\alpha}(S) \stackrel{\text{def}}{=} \lambda i \in [1, n]. \alpha(S \cap \gamma(P_i))$$

nous abstrayons de manière indépendante un morceau de S dans chaque $\gamma(P_i)$, $i \in [1, n]$

nous avons une **correspondance de Galois** pour $\tilde{\mathcal{E}}^\sharp$, s'il en existe une pour $\mathcal{E}^\sharp$

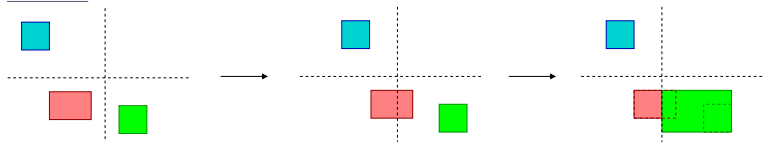
Opérateurs abstraits

Opérateurs abstraits : extension **point à point** de $\mathcal{E}^\#$ en $[1, n] \rightarrow \mathcal{E}^\#$

- $A^\# \widetilde{\cup}^\# B^\# \stackrel{\text{def}}{=} \lambda i \in [1, n]. A^\#(i) \cup^\# B^\#(i)$
- $A^\# \widetilde{\cap}^\# B^\# \stackrel{\text{def}}{=} \lambda i \in [1, n]. A^\#(i) \cap^\# B^\#(i)$
- $A^\# \widetilde{\vee}^\# B^\# \stackrel{\text{def}}{=} \lambda i \in [1, n]. A^\#(i) \vee^\# B^\#(i)$
- $\widetilde{C}^\# \llbracket e_1 \bowtie e_2 \rrbracket A^\# \stackrel{\text{def}}{=} \lambda i \in [1, n]. C^\# \llbracket e_1 \bowtie e_2 \rrbracket A^\#(i)$
- $\widetilde{S}^\# \llbracket V \leftarrow e \rrbracket A^\#$ est **plus complexe**

une image $S^\# \llbracket V \leftarrow e \rrbracket A^\#(i)$ peut échapper de sa partition originale P_i ;
 nous devons couper le résultat abstrait selon les bords des partitions,
 puis fusionner tous les morceaux qui tombent dans la même partition

Exemple : $X \leftarrow X + 2$



$$\widetilde{S}^\# \llbracket V \leftarrow e \rrbracket A^\# \stackrel{\text{def}}{=} \lambda i \in [1, n]. \cup^\# \{ P_i \cap^\# S^\# \llbracket V \leftarrow e \rrbracket A^\#(j) \mid j \in [1, n] \}$$

Exemple d'analyse

Exemple

```

X ← rand(10, 20);
Y ← rand(0, 1);
if Y > 0 then X ← -X;
• Z ← 100/X

```

Analyse :

- $\mathcal{E}^\#$ est le domaine des intervalles
- nous partitionnons par rapport au **signe** de X

$P \stackrel{\text{def}}{=} \{ P^+, P^- \}$ où

$P^+ \stackrel{\text{def}}{=} [0, +\infty] \times \mathbb{Z} \times \mathbb{Z}$ et $P^- \stackrel{\text{def}}{=} [-\infty, 0] \times \mathbb{Z} \times \mathbb{Z}$

- en •, nous trouvons :

$P^+ \mapsto [X \in [10, 20], Y \mapsto [0, 0], Z \mapsto [0, 0]]$

$P^- \mapsto [X \in [-20, -10], Y \mapsto [1, 1], Z \mapsto [0, 0]]$

$\implies$ **pas de division par zéro**

Les analyses par ensemble des parties et par partitionnement d'états donnent des résultats similaires, mais les domaines ont des structures bien différentes : $\mathcal{P}(\mathcal{E}^\#)$ et $[1, n] \rightarrow \mathcal{E}^\#$.

Partitionnement de traces

Sensibilité au chemin d'exécution

Principe : partitionner vis à vis de l'**histoire des calculs**

- garder **séparés** les éléments abstraits issus de **chemins d'exécution différents**
e.g., branches différentes prises, nombre d'itérations de boucle différents, etc.
- **éviter** de fusionner avec $\cup^\sharp$ des éléments :
 - à la fin d'un **if ... then ... else**
 - à la fin d'une itération de boucle
 car l'histoire des calculs est différente

Intuition : vision par transformation de programme

```
X ← rand(-50, 50);
if X ≥ 0 then
  Y ← X + 10
else
  Y ← X - 10;
assert Y ≠ 0
```

→

```
X ← rand(-50, 50);
if X ≥ 0 then
  Y ← X + 10;
  assert Y ≠ 0
else
  Y ← X - 10;
  assert Y ≠ 0
```

l'**assert** n'est plus testé après la fusion des deux branches, mais dans le contexte de chaque branche
 ⇒ amélioration de la précision

le domaine des intervalles échoue à prouver l'assertion à gauche, mais réussit à droite

Domaine abstrait

Formalisation comme domaine abstrait : restreint au cas des tests `if ... then ... else`

- $\mathcal{L}$ dénote un ensemble de **points syntaxiques** du programme correspondants aux instructions $\ell : \text{if } \dots \text{ then } \dots \text{ else}$, où $\ell \in \mathcal{L}$
- l'**histoire abstraite** est un élément de $\mathbb{H} \stackrel{\text{def}}{=} \mathcal{L} \rightarrow \{\text{true}, \text{false}, \perp\}$
 $H \in \mathbb{H}$ se souvient, pour chaque `if ... then ... else` si nous avons exécuté la branche **then** ou **else** lors de la **dernière exécution du test**
 - $H(\ell) = \text{true}$: nous avons pris la branche **then**
 - $H(\ell) = \text{false}$: nous avons pris la branche **else**
 - $H(\ell) = \perp$: nous n'avons **jamais** exécuté ce test

Notes :

- $\mathbb{H}$ se souvient de l'exécution de plusieurs tests successifs
 $\ell_1 : \text{if } \dots \text{ then } \dots \text{ else}; \ell_2 : \text{if } \dots \text{ then } \dots \text{ else}$
- si le test apparaît dans une boucle, $\mathbb{H}$ se souvient uniquement du dernier tour de boucle
`while ... do  $\ell : \text{if } \dots \text{ then } \dots \text{ else}$`
- $\mathbb{H}$ peut être étendu à des histoires de calcul plus longues par
 $\mathbb{H} = (\mathcal{L} \rightarrow \{\text{true}, \text{false}, \perp\})^*$
- $\mathbb{H}$ peut être étendu pour distinguer les itérations de boucle par $\mathbb{H} = \mathcal{L} \rightarrow \mathbb{N}$
- $\mathcal{E}^\# \stackrel{\text{def}}{=} \mathbb{H} \rightarrow \mathcal{E}^\#$
 un élément abstrait différent est associé à chaque histoire abstraite différente

Opérateurs abstraits

- $\check{\mathcal{E}}^\# \stackrel{\text{def}}{=} \mathbb{H} \rightarrow \mathcal{E}^\#$
- $\check{\gamma}(A^\#) = \bigcup \{ \gamma(A^\#(H)) \mid H \in \mathbb{H} \}$
- $\check{\sqsubseteq}, \check{\cup}^\#, \check{\cap}^\#, \check{\vee}$ sont définis **point à point**
- $\check{S}^\# \llbracket V \leftarrow e \rrbracket$ et $\check{C}^\# \llbracket e_1 \bowtie e_2 \rrbracket$ sont également **point à point**
- $\check{S}^\# \llbracket \ell : \text{if } c \text{ then } s_1 \text{ else } s_2 \rrbracket A^\#$ est plus complexe
 - nous commençons par **fusionner** toutes les partitions distinguant ℓ

$$C^\# = \lambda H. A^\#(H[\ell \mapsto \text{true}]) \cup^\# A^\#(H[\ell \mapsto \text{false}]) \cup^\# A^\#(H[\ell \mapsto \perp])$$

en effet, nous ne pouvons nous souvenir que de la dernière exécution de ℓ ;
il faut oublier l'exécution précédente de ℓ pour garder l'exécution courante
 - nous calculons la **branche then**, avec $H(\ell) = \text{true}$

$$T'^\# = \check{S}^\# \llbracket s_1 \rrbracket (\check{C}^\# \llbracket c \rrbracket T^\#)$$
 où

$$T^\# = \lambda H. C^\#(H) \text{ si } H(\ell) = \text{true}, \perp \text{ sinon}$$
 - nous calculons la **branche else**, avec $H(\ell) = \text{false}$

$$F'^\# = \check{S}^\# \llbracket s_2 \rrbracket (\check{C}^\# \llbracket \neg c \rrbracket F^\#)$$
 où

$$F^\# = \lambda H. C^\#(H) \text{ si } H(\ell) = \text{false}, \perp \text{ sinon}$$
 - nous **fusionnons** les deux branches : $T'^\# \check{\cup}^\# F'^\#$
l'union dans $\mathcal{E}^\#$ est en réalité **exacte**
car $\forall H \in \mathbb{H}$: , soit $T'^\#(H) = \perp$, soit $F'^\#(H) = \perp$

Exemple d'analyse

Exemple

```

X ← rand(10, 20);
Y ← rand(0, 1);
ℓ : if Y > 0 then X ← -X;
Z ← 100/X

```

Analyse :

- $\mathcal{E}^\#$ est le domaine des intervalles
- avant le test, l'élément abstrait est $[\ell \mapsto \perp] \mapsto (X \in [10, 20], Y \in [0, 1])$
- après le test, nous avons :
 - $[\ell \mapsto \text{true}] \mapsto (X \in [-20, -10], Y \in [1, 1])$
 - $[\ell \mapsto \text{false}] \mapsto (X \in [10, 20], Y \in [0, 0])$
- ni $X \in [10, 20]$, ni $X \in [-20, -10]$ ne provoque de division par zéro
 $\implies \check{S}^\# \llbracket Z \leftarrow 100/X \rrbracket X^\#$ n'a **pas de division par zéro**

Les analyses par partitionnement d'états et par partitionnement de traces donnent des résultats similaires, mais le critère de partitionnement est différent, et ce ne sont donc pas les mêmes opérations qui mettent à jour les partitions : $X \leftarrow e$ pour le partitionnement d'état, et **if then else** pour le partitionnement de traces.

Un exemple plus complexe [★]_{★★}

Interpolation affine par morceaux

```

X ← rand( TX[0], TX[N]);
I ← 0;
while I < N ∧ X > TX[I + 1] do
  I ← I + 1;
done;
Y ← TY[I] + (X - TX[I]) × TS[I]

```

Sémantique concrète :

interpolation dans une table, par cas selon la valeur de X

- recherche de l'indice I dans la table d'interpolation :
 $TX[I] \leq X \leq TX[I + 1]$
- calcul affine avec pour valeur de base $TY[I]$ quand $X = TX[I]$,
 et pour pente $TS[I]$

Analyse : dans le domaine des intervalles

- sans partitionnement :
 $Y \in [\min TY, \max TY] + (X - [\min TX, \max TX]) \times [\min TS, \max TS]$
- avec partitionnement vis à vis du **nombre d'itérations de boucle** :
 $Y \in \bigcup_{I \in [0, N]} TY[I] + ([0, TX[I + 1] - TX[I]) \times TS[I]$
plus précis car nous gardons la relation entre les indices dans la table

Conseils d'implantation OCaml pour le projet

Produit réduit

But :

- définir un mécanisme de réduction générique pour tout `VALUE_DOMAIN`,
- appliquer à la réduction entre intervalles et parités.

Note : nous raisonnons dans les domaines de valeurs $\mathcal{P}(\mathbb{Z})$, avant l'extension aux domaines d'environnements non-relationnels dans $\mathcal{P}(\mathcal{E})$!

Étapes :

- programmer le module `Parity` de signature `VALUE_DOMAIN` ;
(facile !)
- définir la signature des réductions `REDUCTION`, contenant :
 - deux domaines de signature `VALUE_DOMAIN` ;
 - un opérateur de réduction entre les domaines ;
- définir un foncteur général de produit réduit :
`ReducedProduct` : `REDUCTION` $\rightarrow$ `VALUE_DOMAIN` ;
- définir un module `ParityIntervalReduction` de signature `REDUCTION` contenant la réduction entre parités et intervalles ;
- tout brancher ensemble pour obtenir l'analyse demandée.

Produit réduit : signature des réductions

domains/value_reduction.ml

```
module type VALUE_REDUCTION =  
sig  
  
  module A : VALUE_DOMAIN  
  module B : VALUE_DOMAIN  
  
  type t = A.t * B.t  
  
  val reduce: t -> t  
  
end
```

Produit réduit : instance de réduction

domains/parity_interval_reduction.ml

```
module ParityIntervalsReduction =  
  (struct  
  
    module A = Parity  
    module B = Intervals  
  
    type t = A.t * B.t  
  
    let reduce ((a,b):t) : t =  
      ...  
  
  end : VALUE_REDUCTION)
```

Produit réduit : produit réduit générique

domains/value_reduced_product.ml

```
module ReducedProduct(R : VALUE_REDUCTION) =  
(struct  
  
  module A = R.A  
  module B = R.B  
  
  type t = R.t (* A.t * B.t *)  
  
  let top = ...  
  
  let unary ((a,b):t) (op:int_unary_op) : t =  
    R.reduce ...  
  
  ...  
  
end : VALUE_DOMAIN)
```